PRACTICAL 11

AIM: Program on Calculator
Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:orientation="vertical"

    android:background="#F5F5F5">

    <!-- Header -->

    <TextView

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:text="Calculator"

        android:textColor="#FFF"

        android:textSize="28sp"

        android:textStyle="bold"

        android:background="#3F51B5"

        android:padding="16dp" />

    <!-- Display Area -->

    <LinearLayout

        android:layout_width="match_parent"

        android:layout_height="0dp"

        android:layout_weight="1"

        android:orientation="vertical"

        android:padding="16dp"

        android:background="#FFF">

        <TextView

            android:id="@+id/tvExpression"

            android:layout_width="match_parent"

            android:layout_height="wrap_content"
```

```
        android:textSize="32sp"
        android:textColor="#000"
        android:gravity="end"
        android:paddingBottom="8dp" />
    <View
        android:layout_width="match_parent"
        android:layout_height="2dp"
        android:background="#E91E63" />
    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="0"
        android:textSize="48sp"
        android:textColor="#3F51B5"
        android:gravity="bottom|end"
        android:paddingTop="16dp" />
</LinearLayout>
<!-- Buttons Grid -->
<GridLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="4"
    android:rowCount="5"
    android:padding="8dp">
    <Button android:id="@+id/btn1" style="@style/CalcButton" android:text="1" />
    <Button android:id="@+id/btn2" style="@style/CalcButton" android:text="2" />
    <Button android:id="@+id/btn3" style="@style/CalcButton" android:text="3" />
    <Button android:id="@+id/btnPlus" style="@style/CalcButton" android:text="+" />
```

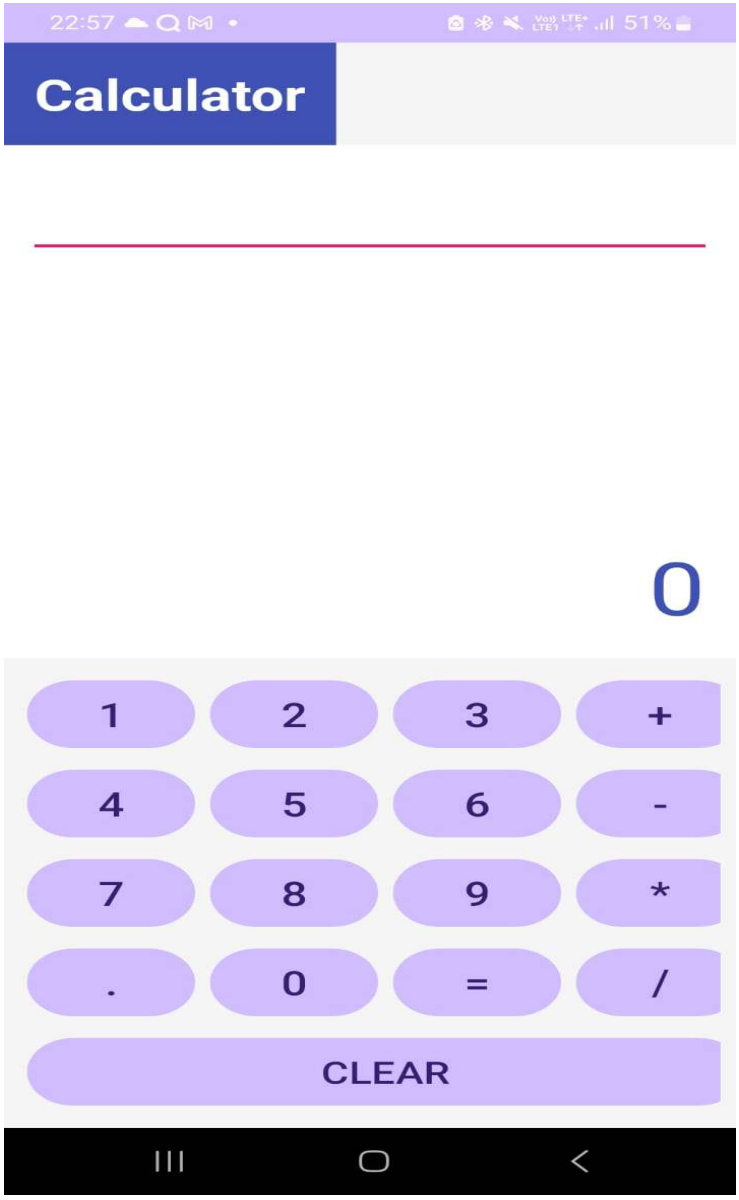
```
        <Button android:id="@+id/btn4" style="@style/CalcButton" android:text="4" />
        <Button android:id="@+id/btn5" style="@style/CalcButton" android:text="5" />
        <Button android:id="@+id/btn6" style="@style/CalcButton" android:text="6" />
        <Button android:id="@+id/btnMinus" style="@style/CalcButton" android:text="-" />
        <Button android:id="@+id/btn7" style="@style/CalcButton" android:text="7" />
        <Button android:id="@+id/btn8" style="@style/CalcButton" android:text="8" />
        <Button android:id="@+id/btn9" style="@style/CalcButton" android:text="9" />
        <Button android:id="@+id/btnMultiply" style="@style/CalcButton" android:text="*" />
        <Button android:id="@+id/btnDot" style="@style/CalcButton" android:text="." />
        <Button android:id="@+id/btn0" style="@style/CalcButton" android:text="0" />
        <Button android:id="@+id/btnEquals" style="@style/CalcButton" android:text="=" />
        <Button android:id="@+id/btnDivide" style="@style/CalcButton" android:text="/" />
        <Button
            android:id="@+id/btnClear"
            style="@style/CalcButton"
            android:text="CLEAR"
            android:textSize="20sp"
            android:layout_columnSpan="4"
            android:layout_gravity="fill_horizontal" />
    </GridLayout>
</LinearLayout>
```

MainActivity.kt

```
package com.example.myapplication
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import java.text.DecimalFormat
class MainActivity : AppCompatActivity() {
    private lateinit var tvExpression: TextView
```

```
private lateinit var tvResult: TextView
private var expression = ""
private var current = ""
private var operator = ""
private var first = 0.0
private var isNew = true
private val df = DecimalFormat("#.##########")
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    tvExpression = findViewById(R.id.tvExpression)
    tvResult = findViewById(R.id.tvResult)
    tvResult.text = "0"
    // Number buttons
    listOf(R.id.btn0, R.id.btn1, R.id.btn2, R.id.btn3, R.id.btn4,
           R.id.btn5, R.id.btn6, R.id.btn7, R.id.btn8, R.id.btn9)
        .forEach { findViewById<Button>(it).setOnClickListener { v ->
            addToExpression((v as Button).text.toString())
        }}
    // Operators
    mapOf(R.id.btnPlus to "+", R.id.btnMinus to "-",
          R.id.btnMultiply to "*", R.id.btnDivide to "/")
        .forEach { (id, op) -> findViewById<Button>(id).setOnClickListener { setOperator(op) }}
    findViewById<Button>(R.id.btnDot).setOnClickListener { addDot() }
    findViewById<Button>(R.id.btnEquals).setOnClickListener { calculate() }
    findViewById<Button>(R.id.btnClear).setOnClickListener { clear() }
}
private fun addToExpression(num: String) {
    if (isNew) { expression = ""; current = ""; isNew = false }
    current += num
    expression += num
```

```
    update()
}
private fun addDot() {
    if (isNew) { current = "0."; expression = "0."; isNew = false }
    else if (!current.contains(".")) {
        current = if (current.isEmpty()) "0." else "$current."
        expression += if (current == "0.") "0." else "."
    }
    update()
}
private fun setOperator(op: String) {
    when {
        current.isNotEmpty() -> {
            if (operator.isNotEmpty()) calculate(false)
            else first = current.toDouble()
            operator = op
            expression += " $op "
            current = ""
            isNew = false
        }
        operator.isNotEmpty() && !isNew -> {
            expression = expression.substringBeforeLast(" ").substringBeforeLast(" ") + " $op "
            operator = op
        }
        isNew -> {
            expression = "${df.format(first)} $op "
            operator = op
            current = ""
            isNew = false
        }
    }
```

```
    update()
}
private fun calculate(final: Boolean = true) {
    if (current.isEmpty() || operator.isEmpty()) return
    val second = current.toDoubleOrNull() ?: return
    val result = when (operator) {
        "+" -> first + second
        "-" -> first - second
        "*" -> first * second
        "/" -> if (second != 0.0) first / second else Double.NaN
        else -> second
    }
    if (result.isNaN()) { tvResult.text = "Error"; clear(); return }
    tvResult.text = df.format(result)
    first = result
    if (final) { isNew = true; expression = df.format(result) }
    else { current = ""; operator = "" }
}
private fun clear() {
    expression = ""; current = ""; operator = ""; first = 0.0; isNew = true
    tvExpression.text = ""; tvResult.text = "0"
}
private fun update() {
    tvExpression.text = expression
    if (operator.isNotEmpty() && current.isNotEmpty()) {
        current.toDoubleOrNull()?.let { second ->
            val preview = when (operator) {
                "+" -> first + second
                "-" -> first - second
                "*" -> first * second
                "/" -> if (second != 0.0) first / second else null
```

```
                else -> null
            }
            preview?.let { tvResult.text = df.format(it) }
        }
    }
  }
}
```